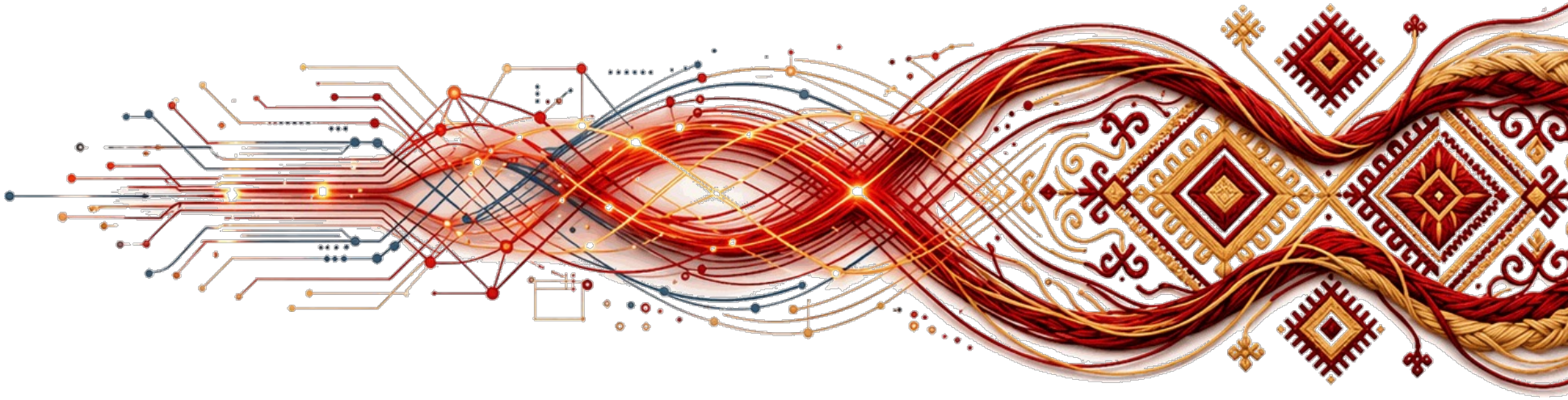
Introduction to Data Science

BASIC TRACK · LECTURE 5



Supervised learning II: classification, trees and ensembles

Logistic regression, decision trees, random forests and boosting

Prof. Dimitar Trajanov, PhD
Prof. Igor Mishkovski, PhD
Prof. Andrea Kulakov, PhD
Prof. Slobodan Kalajdziski, PhD

"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Vezilka
EuroHPC Joint Undertaking
Co-funded by the European Union
Republic of North Macedonia
Ministry of Digital Transformation

Agenda

1. **Classification and logistic regression**
   Probabilities, odds, the confusion matrix
2. **Decision trees**
   Impurity, information gain, the restaurant example
3. **Model complexity and pruning**
   Tree depth, cost-complexity pruning
4. **Bagging and random forests**
   Bootstrap samples, the out-of-bag error
5. **Boosting**
   AdaBoost, gradient boosting, the learning rate
6. **Naive Bayes**
   Counting, smoothing, text classification
7. **Interpretability and a fair comparison**
   Permutation importance, a common protocol

Learning objectives of the lecture

- After this lecture, the student is able to:
  - Explain why logistic regression models a **probability** and interpret its coefficients through the odds.
  - Grow a **decision tree** by greedy splitting with the Gini index or entropy, and prune it by cross-validation.
  - Explain how **bagging and random forests** reduce variance and estimate their error out of bag.
  - Describe **AdaBoost and gradient boosting** and the role of the learning rate.
  - Apply the **naive Bayes** classifier with Laplace smoothing.
  - Compare classifiers on the same folds and metric, and explain a model with **permutation importance**.

01

Classification and logistic regression

Lecture 4 modelled a numerical response; this section models the probability of a class



Classification in everyday words: predicting a category

1 Spam filter

Input: the words of an e-mail.
Output: spam / not spam.

2 Loan default

Input: card balance, income, student.
Output: default yes / no (this lecture).

3 Medical test

Input: symptoms, lab values.
Output: ill / healthy.

4 Digits

Input: the pixels of an image.
Output: one of 0, 1, …, 9.



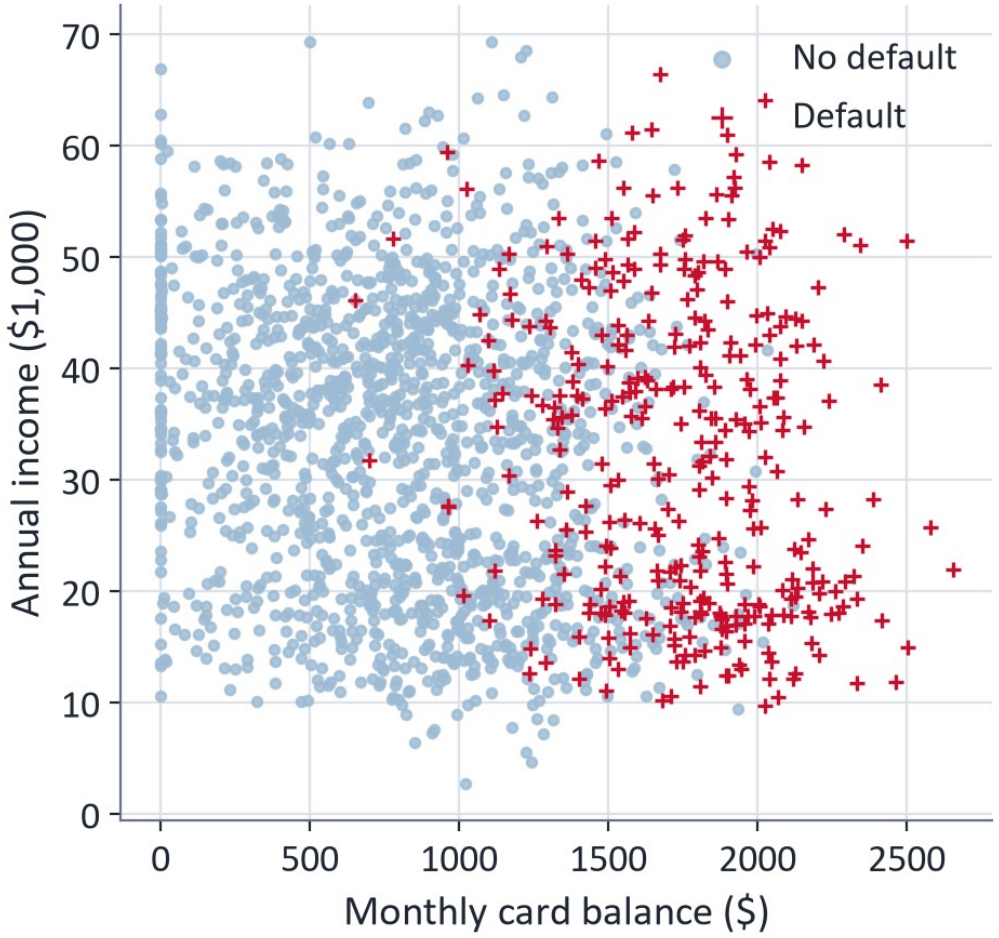
- *Regression* (Lecture 4) predicts a **number**; *classification* predicts a **category**, called a *class* or *label*.
- Most classifiers first estimate a **probability**, e.g. P(default) = 0.12; a **threshold** then turns it into a class.

Key words and symbols used in this lecture

Term	Meaning	Example: Default data
Class (label)	One of the possible answers	default: Yes or No
Positive class	The class we want to detect, coded 1	Yes, the holder defaults
Probability p	Chance of the positive class, between 0 and 1	p = 0.12
Odds p / (1 − p)	How many times more likely Yes than No	p = 0.8 → odds 4 ("4 to 1")
Log-odds	Logarithm of the odds; any real number	odds 4 → log-odds 1.39
Threshold	Cut-off that turns p into a class	predict Yes if p ≥ 0.5
Decision boundary	Where the predicted class changes	balance ≈ $1,936 for threshold 0.5
Confusion matrix	Table of predicted against actual classes	TP, FP, FN, TN
Impurity	How mixed the classes in a tree node are	Gini, entropy
Ensemble	Many models combined into one prediction	random forest, boosting

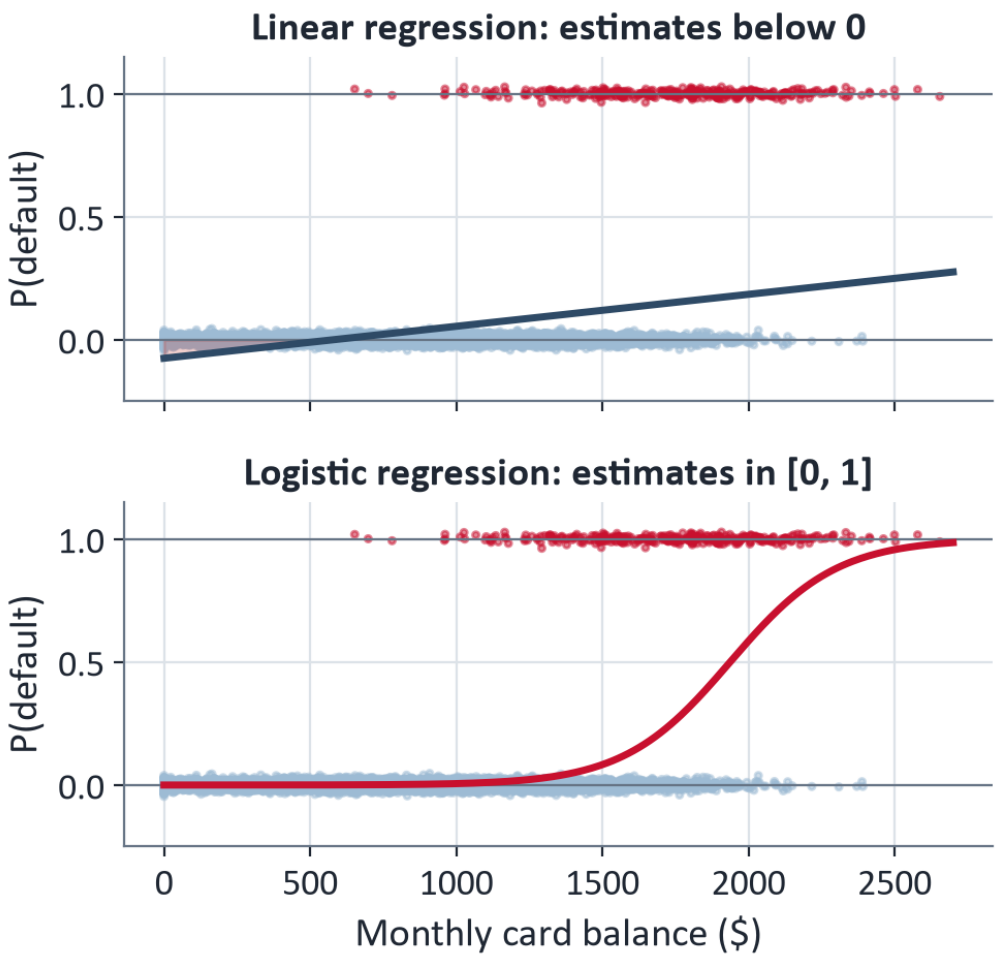
Classification: predicting a categorical response

- In *classification*, the response Y takes values in **a finite set of classes**, and each observation is assigned to one of them.
  - The predictors X may be numerical or categorical, as in regression.
- The ISLR *Default* data record **10,000 card holders**: monthly balance, annual income, student status and default.
  - Only **3.3 %** of the holders default, so the classes are strongly imbalanced.
- Defaulters concentrate at **high balances**, while income separates the classes only weakly.
  - This pattern is descriptive; predictive value requires evaluation on held-out observations.
- Probabilistic classifiers estimate **class probabilities**; other classifiers may provide only a score or a label.
  - The threshold is chosen from the costs of the two types of error, which Lecture 6 treats in detail.



Default data, ISLR (James et al., 2021)

Linear regression on a binary response



Both models fitted to the Default data

- With the coding default = 1 and no default = 0, least squares estimates **P(default | balance)** by a straight line.
- The line predicts **negative probabilities** for 31 % of the holders; extrapolation can also exceed 1, although that is not shown here.
  - A straight line cannot respect the bounds of a probability.
- With more than two unordered classes, a numeric coding imposes **an order and equal spacing** that do not exist.
- Logistic regression models the probability directly, with an S-shaped curve **bounded by 0 and 1**.
  - The curve is steepest where the classes overlap, near a balance of $2,000; flat tails express model estimates, not certainty.
- Both fits use **the same predictor**; only the link between the linear predictor and the probability differs.

The logistic model and the odds

> Logistic regression models the probability p(X) = P(Y = 1 | X) of class 1 in two steps:
>
> $$p(X) = \underbrace{\frac{1}{1 + e^{-z}}}_{\text{S–curve: always between 0 and 1}}, \quad z = \underbrace{\beta_0 + \beta_1 X}_{\text{linear score}}$$
>
> $$\underbrace{\log \frac{p(X)}{1 - p(X)}}_{\text{log–odds}} = \beta_0 + \beta_1 X$$
>
> — After James, Witten, Hastie and Tibshirani (2021), An Introduction to Statistical Learning, Chapter 4

- The *odds* p / (1 − p) range over (0, ∞): p = 0.2 gives odds of **0.25**, and p = 0.9 gives odds of **9**.
- A one-unit increase in X **multiplies the odds by e^β₁**; the coefficients are estimated by maximum likelihood.

Probability, odds and log-odds: one idea, three scales

Probability p	Odds p / (1 − p)	Log-odds	In words
0.1	0.11	−2.20	1 Yes for every 9 No
0.2	0.25	−1.39	1 Yes for every 4 No
0.5	1	0	even chance
0.8	4	+1.39	4 Yes for every 1 No
0.9	9	+2.20	9 Yes for every 1 No
0 to 1	**0 to ∞**	**−∞ to +∞**	**range**

1 **Probability**

p: the chance of Yes

2 **Odds**

p / (1 − p): Yes against No

3 **Log-odds**

log of the odds; 0 when p = 0.5

4 **Why log-odds?**

a line $\beta_0 + \beta_1 x$ can take any value; so can the log-odds

Logistic regression is a straight line on the log-odds scale. A slope β_1 adds β_1 to the log-odds per unit of x, which multiplies the odds by e^{β_1}.

Logistic regression: likelihood and regularisation

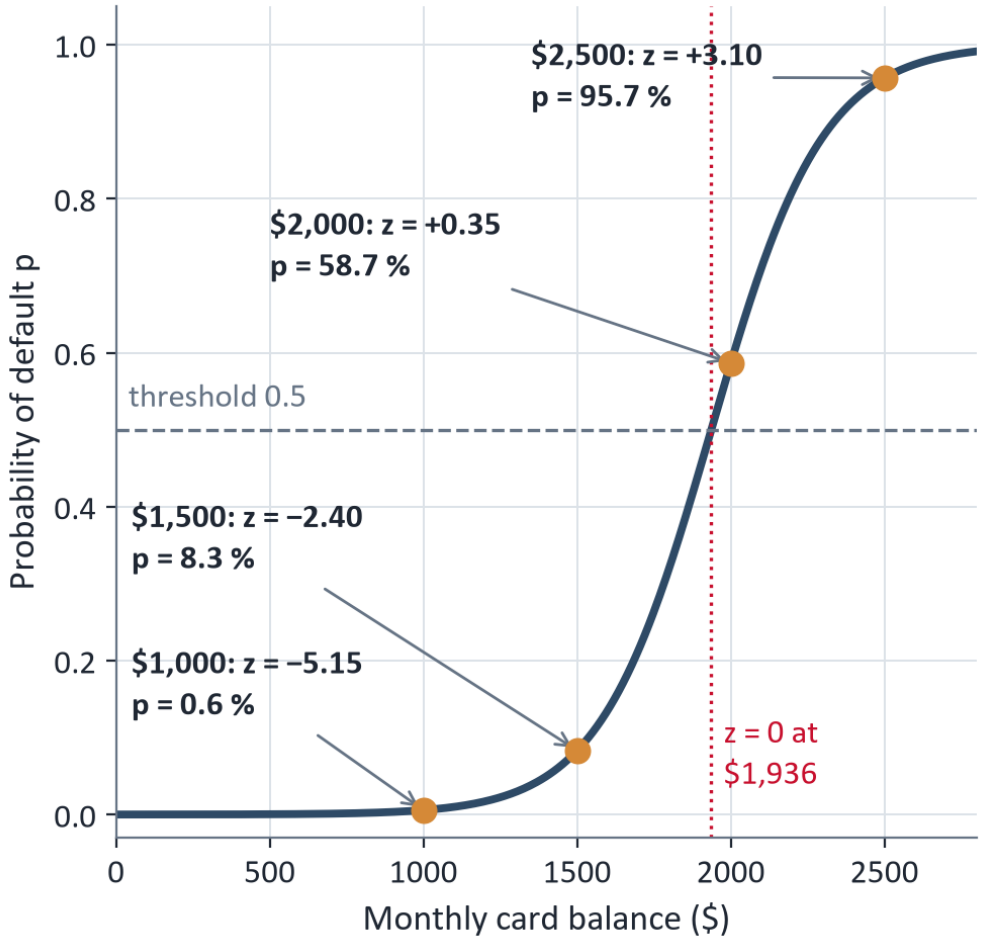
$$\text{log loss} = \frac{1}{n}\sum_{i=1}^{n} \ell_i\,, \quad \ell_i = \underbrace{-\log p_i}_{\text{if y = 1}} \quad \text{or} \quad \underbrace{-\log(1-p_i)}_{\text{if y = 0}}$$



- *Maximum likelihood* picks the β that make the observed classes **most probable**; this is the same as minimising the **log loss** above.
- For a row with y = 1 the loss is −log p: p = 0.9 costs **0.11**, p = 0.1 costs **2.30**.
- **Confident wrong** predictions are punished hardest.
- An *L2 penalty*, as in ridge, keeps the coefficients small; **standardise** the predictors first.
- If the classes are **perfectly separated**, unpenalised coefficients grow without limit; the penalty prevents this.

Worked example: from card balance to probability of default

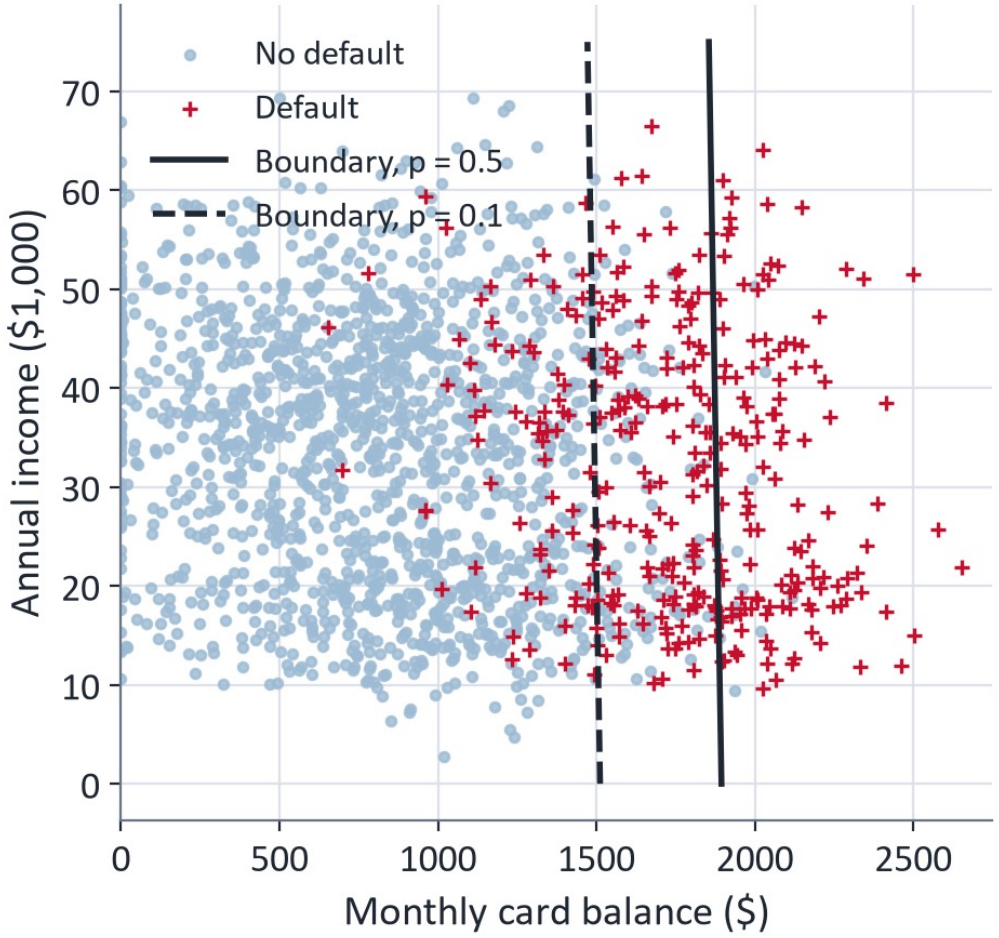
- Fitted on balance alone: **z = −10.65 + 0.0055 · balance**.
- Balance $1,000: z = −10.65 + 5.50 = **−5.15**, p = 1 / (1 + e^5.15) ≈ **0.6 %**.
- Balance $2,000: z = −10.65 + 11.00 = **0.35**, p = 1 / (1 + e^−0.35) ≈ **59 %**.
- Every extra $100 adds 0.55 to the log-odds, so the **odds are multiplied by e^0.55 ≈ 1.73**; the probability does not grow by a fixed amount.
- With threshold 0.5 the model predicts default when z ≥ 0, that is for a balance of about **$1,936** or more.



The logistic curve of the Default model

Multiple logistic regression on the Default data

- Fitted on balance alone, the model gives β_0 = **-10.65** and β_1 = **0.0055** per dollar.
  - The estimated probability of default is 0.58 % at a balance of $1,000 and **58.6 %** at $2,000.
- With balance, income and student status, *multiple logistic regression* is fitted in the same way.
  - Being a student lowers the log-odds by **0.65** at fixed balance and income; this is an association, not a causal effect.
- The *decision boundary* p(X) = c is **a hyperplane** in the original features when the log-odds are linear.
  - Lowering the threshold from 0.5 to 0.1 flags more holders and trades false alarms for missed defaults.
- A curved logistic boundary requires **nonlinear features**, such as squares or products of predictors; trees in Section 02 learn it directly.



Boundaries for non-students, two thresholds

The confusion matrix in plain words

	Predicted: default	Predicted: no default
Actually defaults	**TP = 5** **true positive** **caught a defaulter**	**FN = 5** **false negative** **missed a defaulter**
Actually does not default	**FP = 10** **false positive** **false alarm**	**TN = 80** **true negative** **correctly left alone**

Precision = 5 / 15 = 0.33

Of the 15 holders we flagged, how many really defaulted? Few false alarms → high precision.

Recall = 5 / 10 = 0.50

Of the 10 real defaulters, how many did we find? Few misses → high recall.

- A smoke alarm that rings at every toast has **high recall but low precision**; one that rarely rings has the opposite problem.
- Lowering the threshold catches more defaulters (recall up) but raises more false alarms (precision down); the **costs of the two errors** decide.

Classification metrics and a worked confusion matrix

Actual / predicted	Positive	Negative
Positive	TP = 5	FN = 5
Negative	FP = 10	TN = 80

ROC AUC measures **ranking** across all thresholds: the probability that a random positive scores higher than a random negative.

Log loss and *Brier score* evaluate the **probabilities** rather than one thresholded decision.

F1 combines precision and recall, but **ignores the true negatives**.

Illustrative 100-record sample:

$$\text{Accuracy} = \frac{TP + TN}{n} = \frac{85}{100} = 0.85$$

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{5}{15} = 0.33$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{5}{10} = 0.50$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 0.40$$

- A majority-negative baseline has **0.90 accuracy** in this illustrative sample but zero recall and F1 for the positive class. Metric choice must follow error costs and the deployment task; high ROC AUC alone does not establish calibrated probabilities (scikit-learn model evaluation).

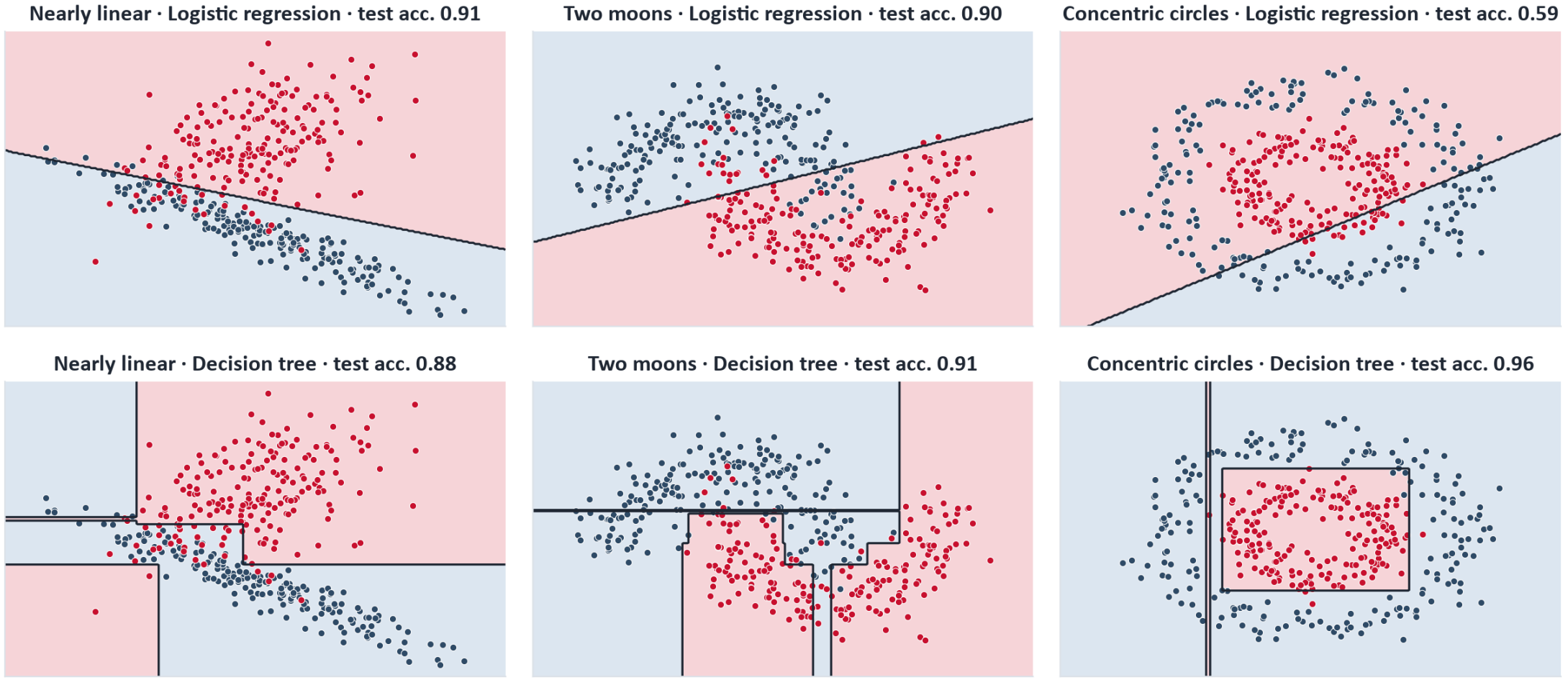
02

Decision trees: geometry and learning

When the classes are separated but the boundary is not a line, a flow chart of simple tests is an interpretable alternative

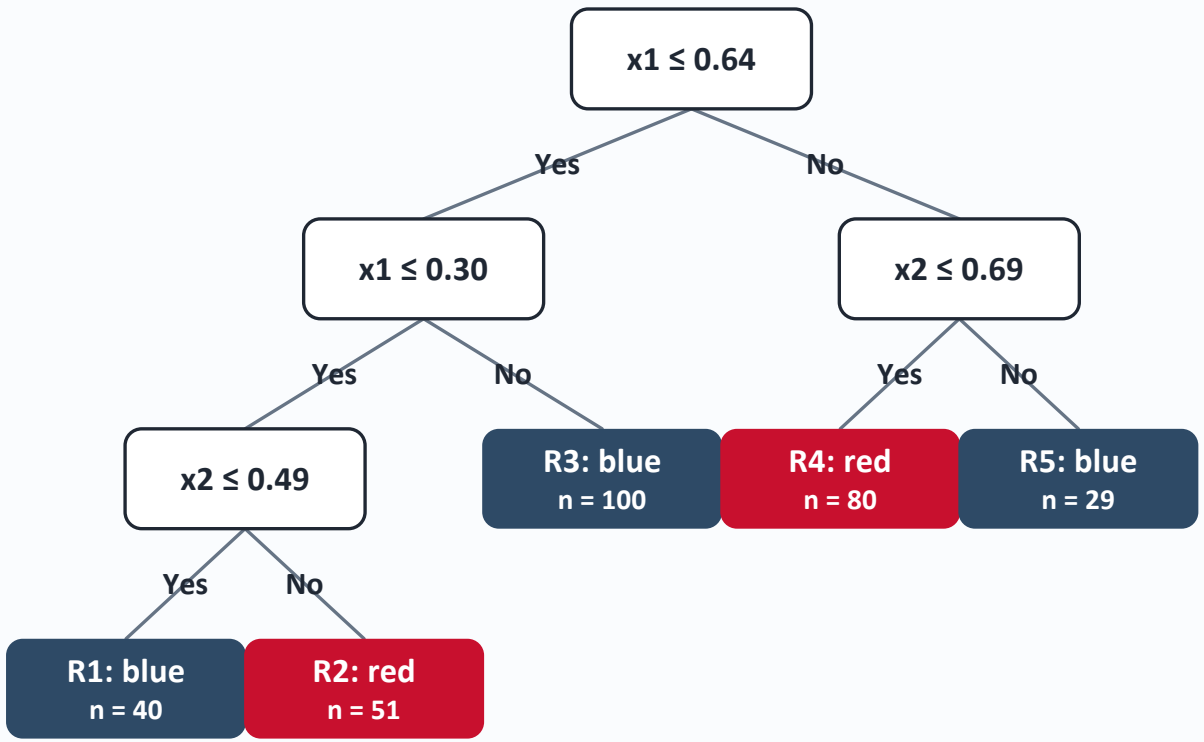


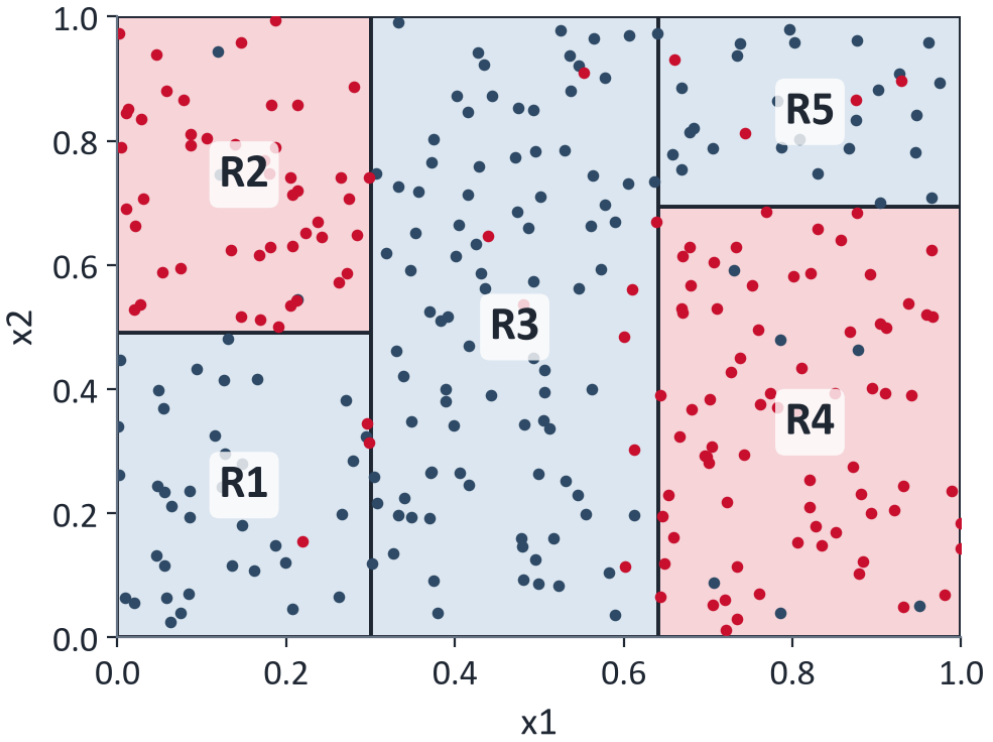
Decision boundaries of linear and tree classifiers



Logistic regression (top) and depth-5 trees (bottom); synthetic data, computed independent test accuracy.

A decision tree as a partition of the feature space

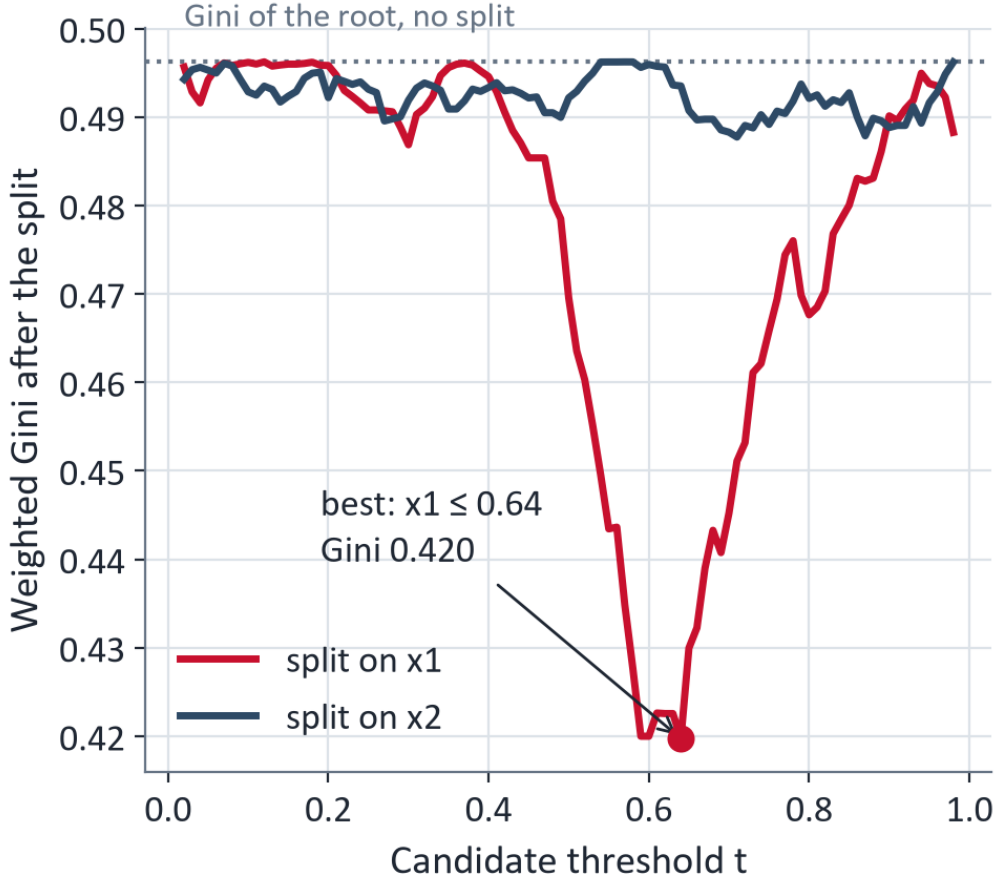




- Each node tests **one predictor against a threshold**. Recursive splits form rectangles R1 … R5; arbitrary rectangular tilings need not have this structure. A leaf predicts **its majority class**; this tree classifies 92 % of the 300 training points correctly.

Learning a tree: the greedy search for a split

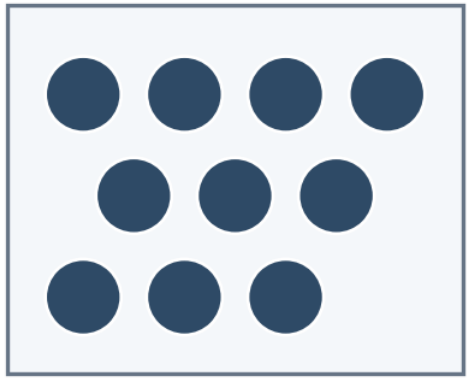
- Optimal tree construction is **computationally difficult**; some optimal-tree decision problems are NP-complete (Hyafil and Rivest, 1976).
  - At each node, every predictor and every threshold between observed values is evaluated.
  - The split with the **lowest weighted impurity** of the two children is kept; the procedure recurses on each child.
- On the data of the previous slide, the root has Gini 0.496; the best plotted grid split, **x1 ≤ 0.64**, lowers it to 0.420.
  - The best split on x2 reaches only 0.488, so x2 is not chosen at the root.
- Greedy splitting is **myopic**: a split that helps only in combination with a later split may never be chosen.
  - Sorting costs O(p · n log n) at a node; cumulative scanning then costs O(p · n). Implementations may reuse sorted values.



One step of the greedy algorithm

Impurity in plain words: how mixed is a node?

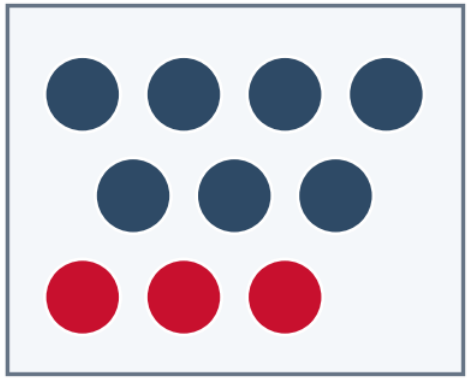
Pure node: 10 blue, 0 red



Gini 0.00 entropy 0.00

error 0.00

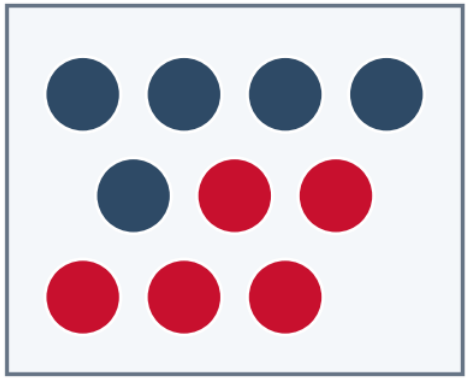
Mostly one class: 7 blue, 3 red



Gini 0.42 entropy 0.88

error 0.30

Evenly mixed: 5 blue, 5 red



Gini 0.50 entropy 1.00

error 0.50

- A node is **pure** when all its rows have the same class: impurity 0. A 50/50 mix is the **most impure**: Gini 0.5, entropy 1 bit.
- A good split produces children that are **purer than the parent**; the tree chooses the split with the **largest drop in impurity**.

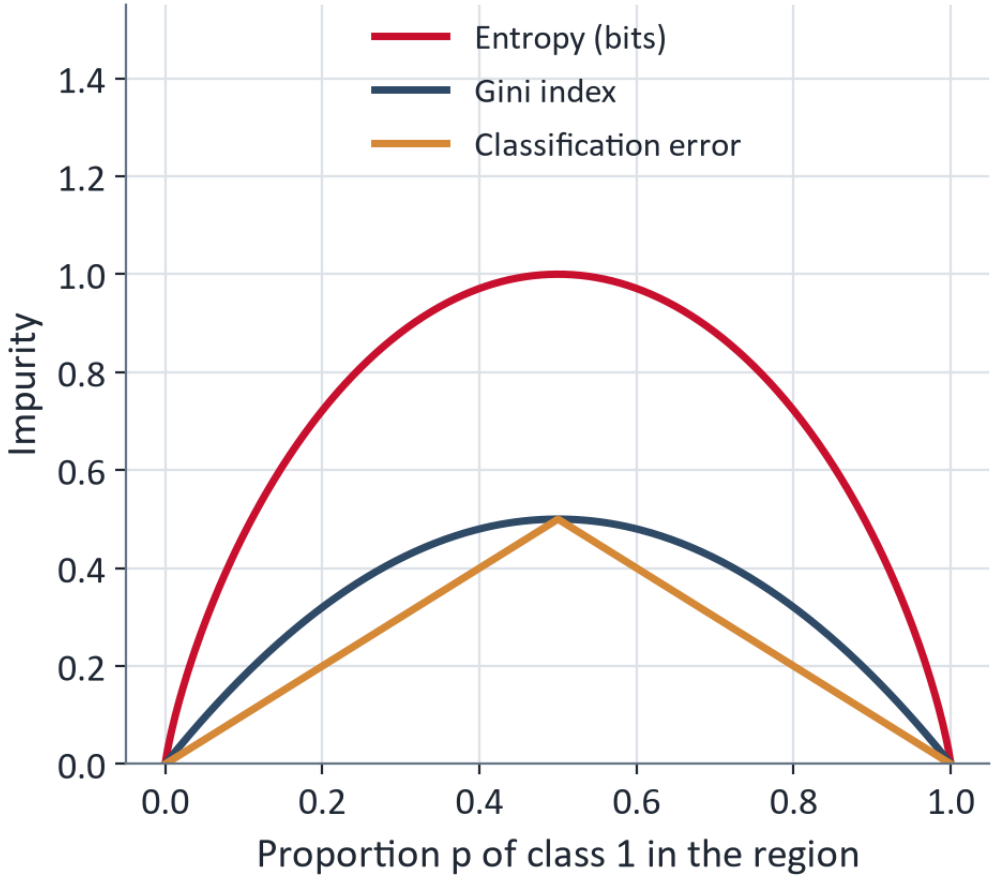
Splitting criteria: error, Gini index and entropy

$$\text{Error} = 1 - \max_k p_k$$

$$\text{Gini} = 1 - \sum_k p_k^2$$

$$\text{Entropy} = -\sum_k p_k \log_2 p_k$$

- p_k is the **share of class k** in the node. All three are **0 for a pure node** and largest for a 50/50 mix.
- *Gini*: the chance that two rows drawn at random from the node have **different classes**.
- *Entropy*: the average number of yes/no questions (bits) needed to **identify the class** of a row.
- Gini and entropy reward **purer children** even when the majority class does not change; the error does not, so it is rarely used to grow trees.
- Both give **similar trees**; Gini is the common default because it needs no logarithm.



Impurity of a two-class region

The restaurant data set: deciding whether to wait

Ex.	Alt	Bar	Fri	Hun	Patrons	Price	Rain	Res	Type	Est	Wait
X1	T	F	F	T	Some	$$$	F	T	French	0–10	Yes
X2	T	F	F	T	Full	$	F	F	Thai	30–60	No
X3	F	T	F	F	Some	$	F	F	Burger	0–10	Yes
X4	T	F	T	T	Full	$	F	F	Thai	10–30	Yes
X5	T	F	T	F	Full	$$$	F	T	French	>60	No
X6	F	T	F	T	Some	$$	T	T	Italian	0–10	Yes
X7	F	T	F	F	None	$	T	F	Burger	0–10	No
X8	F	F	F	T	Some	$$	T	T	Thai	0–10	Yes
X9	F	T	T	F	Full	$	T	F	Burger	>60	No
X10	T	T	T	T	Full	$$$	F	T	Italian	10–30	No
X11	F	F	F	F	None	$	F	F	Thai	0–10	No
X12	T	T	T	T	Full	$	F	F	Burger	30–60	Yes

Table — twelve examples, ten attributes (alternative nearby, bar, Friday or Saturday, hungry, …); after Russell and Norvig (2020).

Worked example: the information gain of Patrons

Branch	Rows	Yes	No	Entropy	Weight	Weight × entropy
None	2	0	2	0.000	2/12	0.000
Some	4	4	0	0.000	4/12	0.000
Full	6	2	4	0.918	6/12	0.459
All	**12**	**6**	**6**	**1.000**		**0.459**

1. **Before the split**
   6 Yes, 6 No: H = 1 bit
2. **Entropy per branch**
   None and Some are pure: 0
3. **Weighted average**
   $0 + 0 + 6/12 \cdot 0.918 = 0.459$
4. **Gain**
   IG = 1 − 0.459 = 0.541 bits

- Entropy of the Full branch (2 Yes, 4 No): $-(2/6)\cdot\log_2(2/6) - (4/6)\cdot\log_2(4/6) =$ **0.918**.
- *Type*: every branch is half Yes and half No (entropy 1), the weighted average stays 1, so IG = **0**: Type tells us nothing.

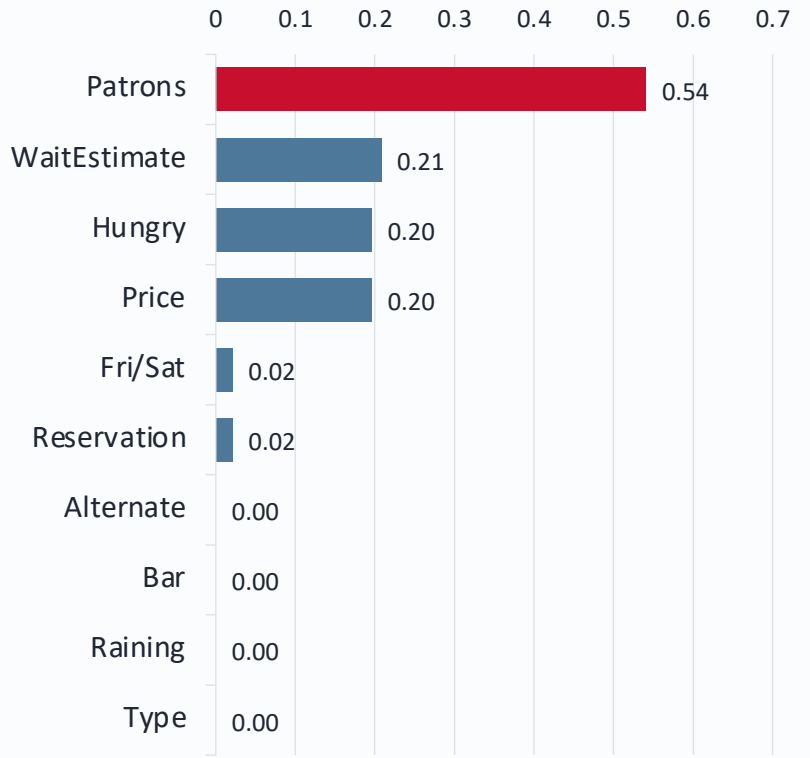
Information gain at the root: Patrons versus Type

Patrons: IG = 0.541

None	Some	Full
Yes —	Yes X1 X3 X6 X8	Yes X4 X12
No X7 X11	No —	No X2 X5 X9 X10

Type: IG = 0.000

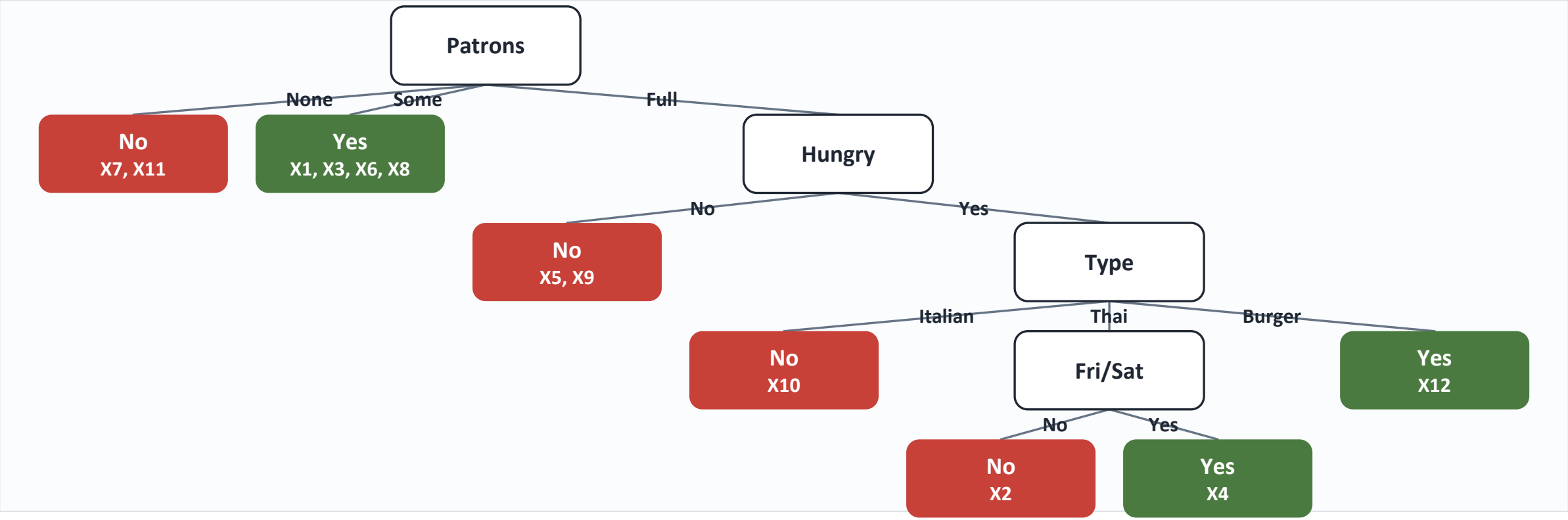
French	Italian	Thai	Burger
Yes X1	Yes X6	Yes X4 X8	Yes X3 X12
No X5	No X10	No X2 X11	No X7 X9



$$IG(A) = \underbrace{H(\text{Wait})}_{\text{impurity before the split}} - \underbrace{\sum_v \frac{n_v}{n} H(\text{Wait} \mid A = v)}_{\text{weighted impurity of the branches after the split}}$$

- Start: 6 Yes and 6 No, so H = **1 bit**.
- *Patrons* leaves two pure branches: IG = **0.541** bits.
- Every *Type* branch stays half Yes: IG = **0**.

The decision tree learned from the restaurant data



- ID3 applies the root step recursively: under Patrons = Full, Hungry gains 0.252 bits (attribute order breaks ties), followed by Type and Fri/Sat. **Four internal tests** classify all twelve training examples. ID3 uses categorical multiway splits; CART uses binary splits, including thresholds (Breiman et al., 1984).

03

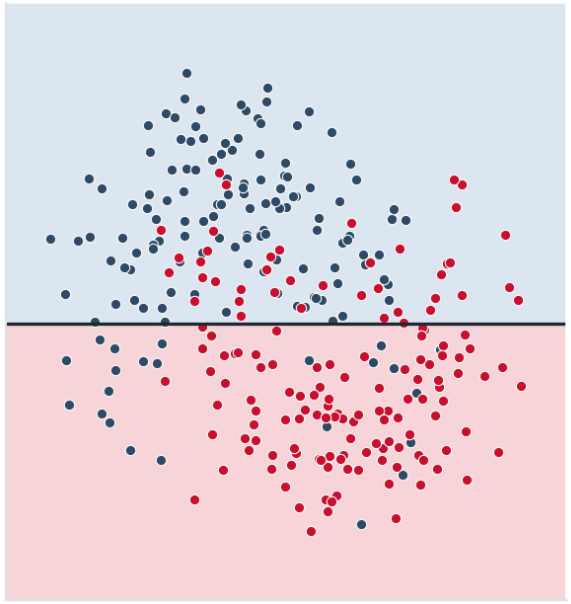
Model complexity, pruning and selection

A tree grown until its leaves are pure memorises the training data; this section controls its size

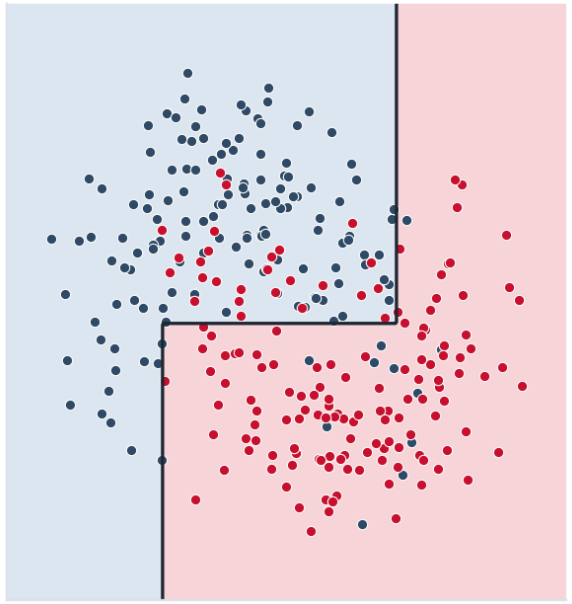


Tree depth and the bias–variance trade-off

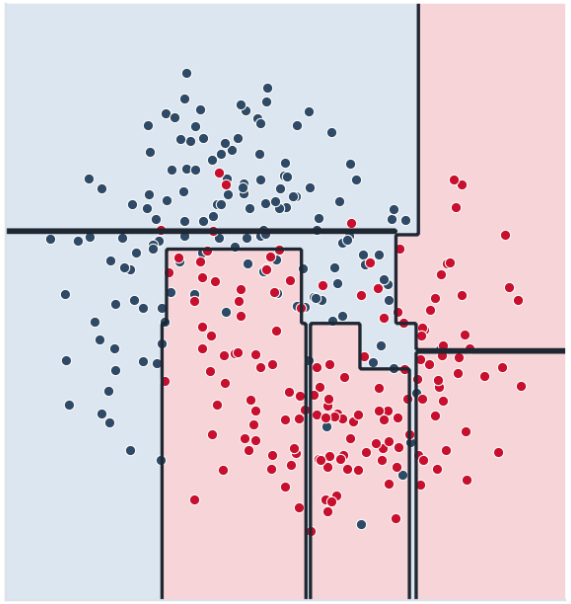
max_depth = 1
train 0.79 · test 0.80 · 2 leaves



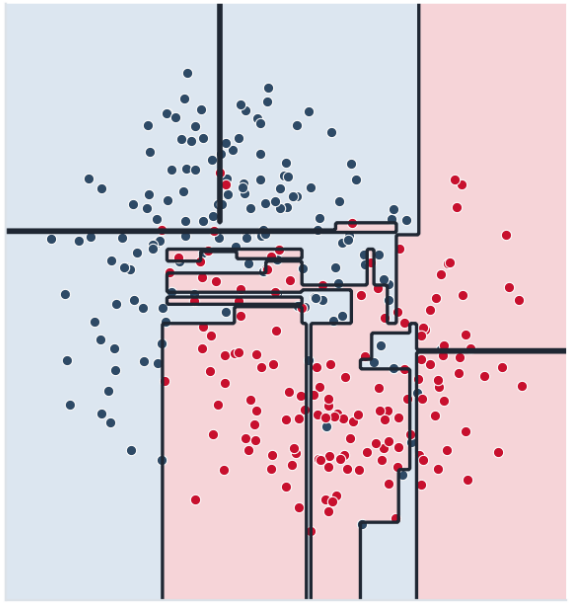
max_depth = 3
train 0.88 · test 0.88 · 7 leaves



max_depth = 6
train 0.92 · test 0.83 · 25 leaves



max_depth = None
train 1.00 · test 0.82 · 52 leaves

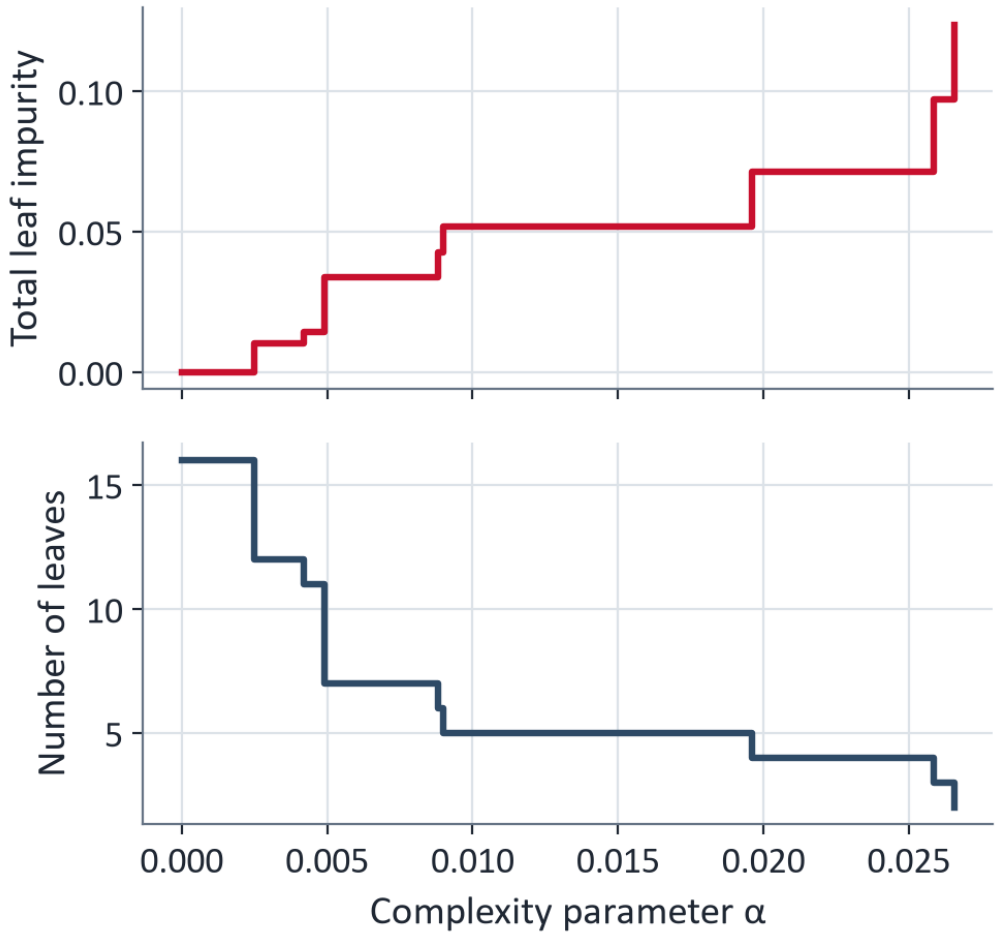


Feature-noisy moons: depth 3 test accuracy 0.879; unrestricted tree: training 1.00, test 0.817.

Cost-complexity pruning

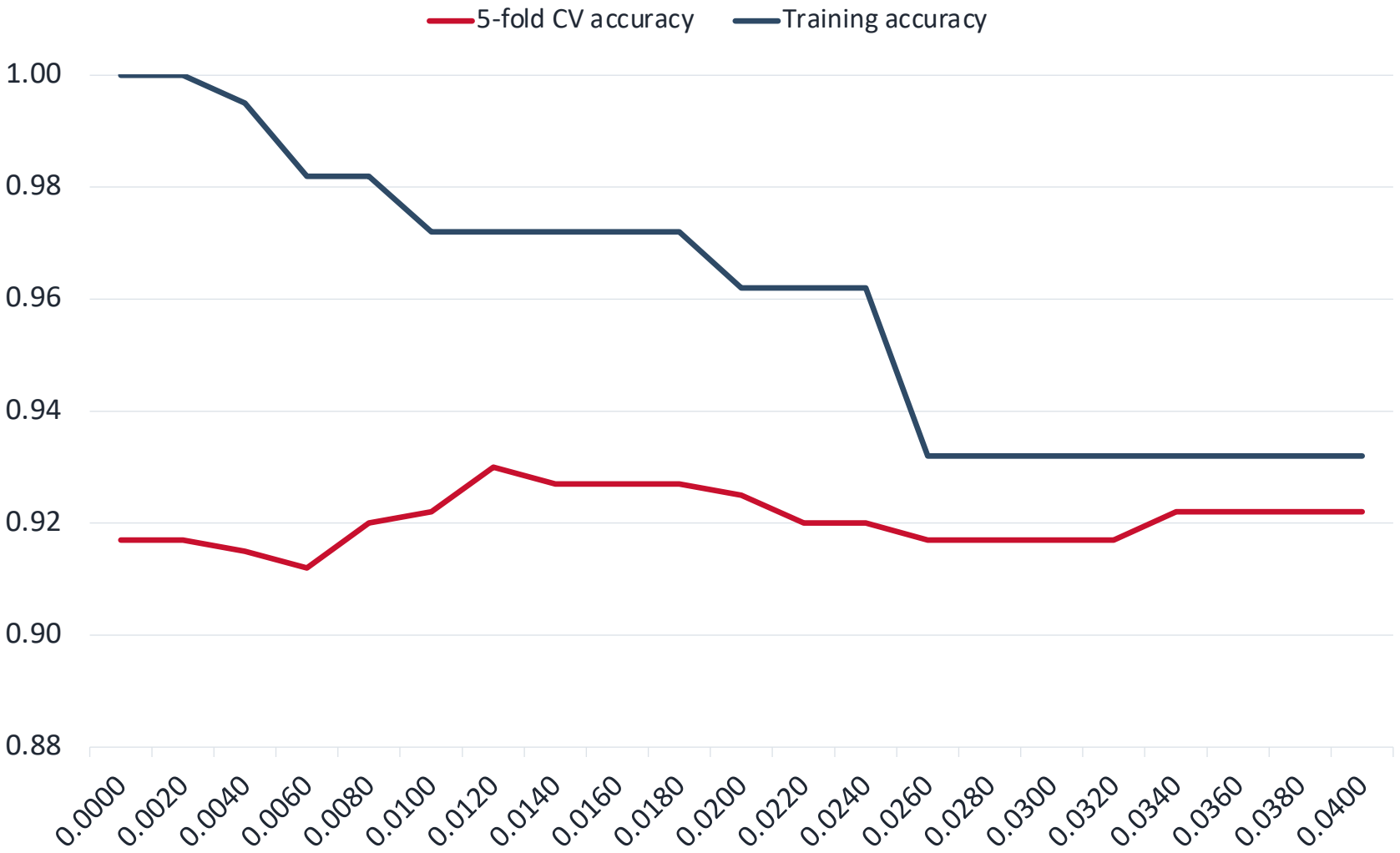
$$\text{minimise} \quad \underbrace{R(T)}_{\text{impurity of the leaves (training)}} + \underbrace{\alpha}_{\text{price}} \cdot \underbrace{|T|}_{\text{number of leaves}}$$

- *Stopping rules* (maximum depth, minimum leaf size) decide **before** growing and may stop too early or too late.
- *Cost-complexity pruning* grows a **large tree first** and then cuts it back; α is the **price of one leaf**.
- α = 0 keeps the full tree; as α grows, the **weakest branches are cut first**, until only the root is left.
- Breast-cancer data: the full tree has **16 leaves** and zero training impurity; the figure shows impurity rising and leaves falling as α grows.
- The best α is chosen by **cross-validation** (next slide).



Pruning path, breast-cancer training set

Selecting the pruning parameter by cross-validation



- The horizontal axis lists a **predeclared α grid**, independent of validation-fold labels.
- Training accuracy **falls** as α removes leaves; the objective balances fit against complexity.
- The standard error across folds at the selected α is **0.017**; correlated folds make it an approximate summary.
- CV accuracy peaks at **α = 0.0120**, a tree of 5 leaves.
- Test accuracy: 0.918 pruned, 0.906 unpruned; pruning does not guarantee improvement on a particular test split.

04

Bagging and random forests

The deep trees of Section 03 have low bias and high variance; averaging reduces variance when errors are not perfectly correlated



Bagging in plain words: many trees, each on its own sample

1 Wisdom of the crowd

Single guesses of a quantity vary a lot; the average of many independent guesses is close.

2 Bootstrap sample

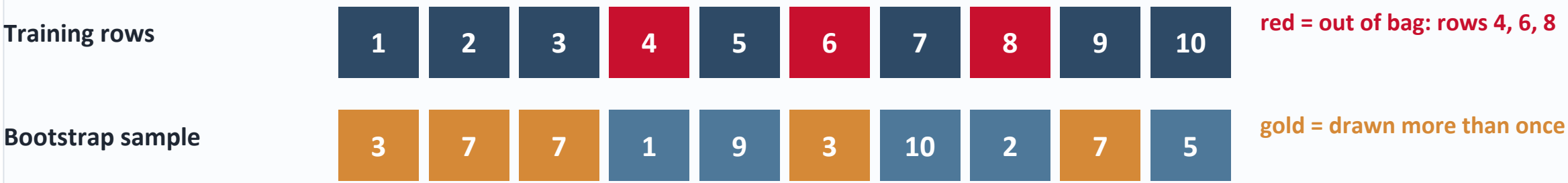
Draw n rows with replacement: some rows appear twice, some not at all.

3 One tree per sample

Each tree sees slightly different data and makes different mistakes.

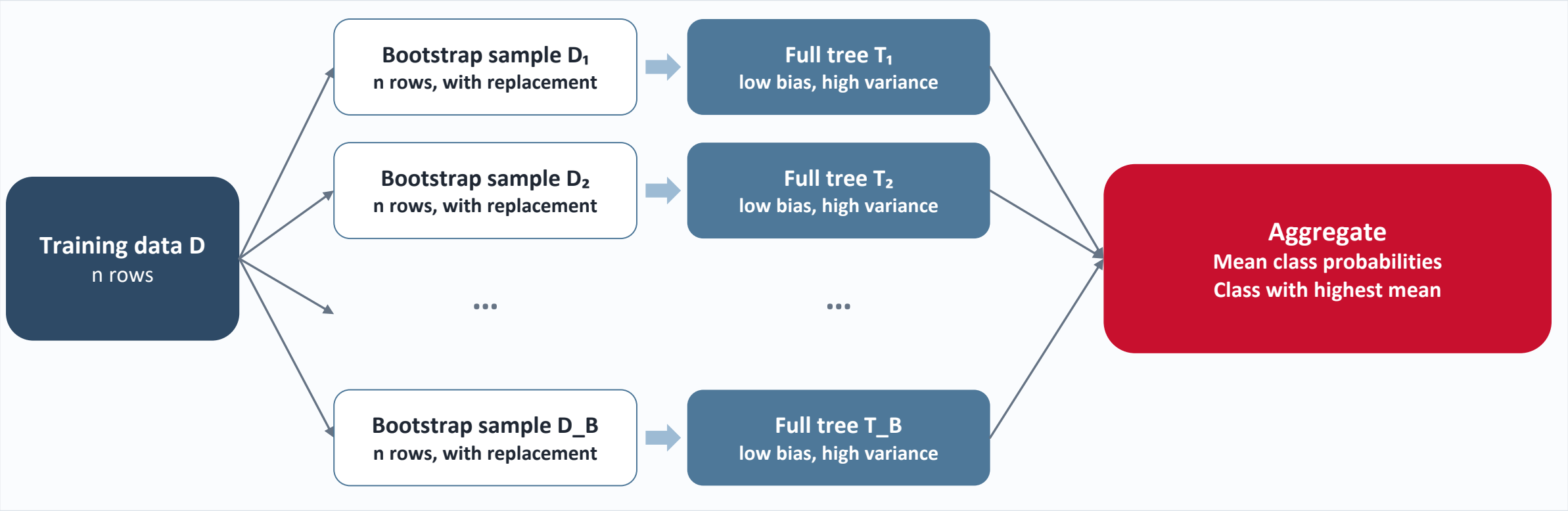
4 Average

Average the trees' probabilities; part of their mistakes cancel out.



- This sample contains 7 different rows; rows 4, 6, 8 are **out of bag** (30 %). On average about **37 %** are.
- Averaging helps most when the trees are **not too similar**; random forests (later in this section) make them less similar on purpose.

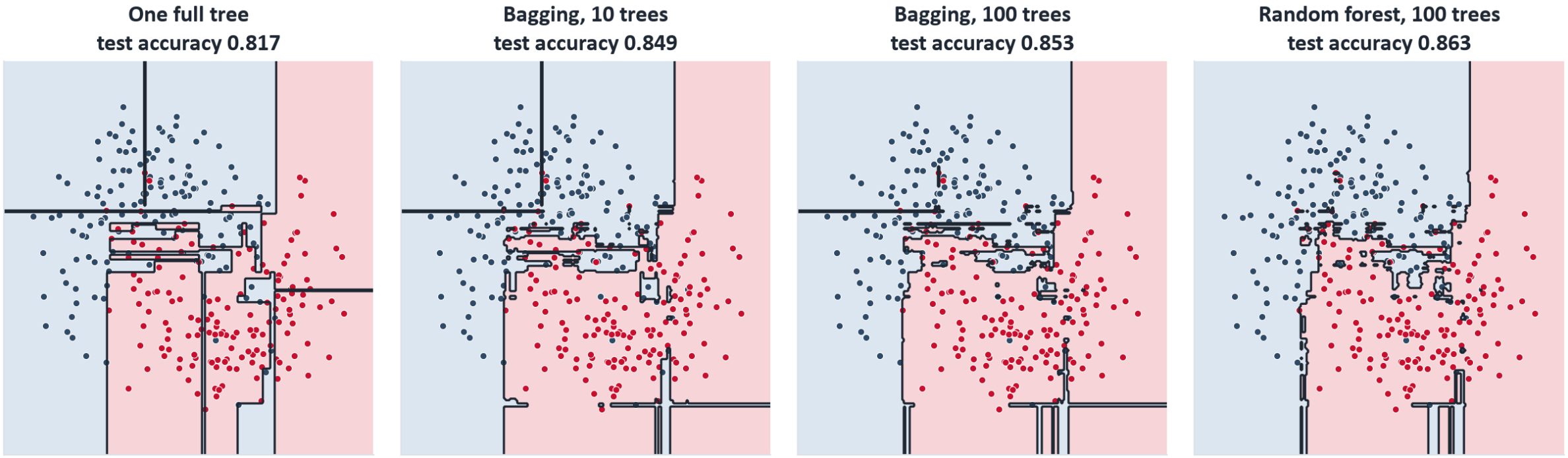
Bootstrap aggregation and prediction averaging



$$\mathrm{Var}(\text{average}) = \underbrace{\rho\sigma^2}_{\text{stays: the trees are correlated}} + \underbrace{\frac{1-\rho}{B}\sigma^2}_{\text{shrinks as the number B of trees grows}}$$

- *Bagging* fits a full tree to each of B bootstrap samples and **averages** their predictions.
- B trees with variance σ^2 and correlation ρ: more trees remove only the **second term**.

Averaging bootstrap trees reduces variance

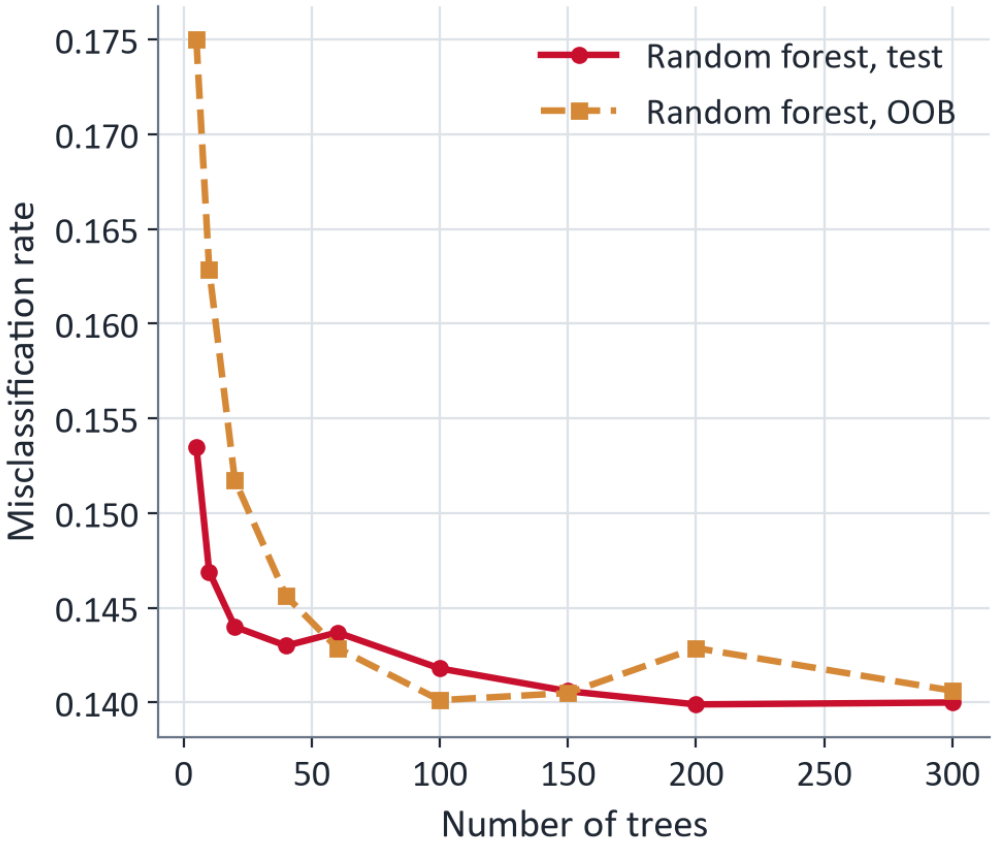


Noisy moons: test accuracy 0.817 (one tree), 0.853 (100 bagged trees), 0.863 (100-tree forest).

The out-of-bag error

$$\underbrace{\left(1-\frac{1}{n}\right)^{n}}_{\text{one row is not drawn in any of n draws}} \approx e^{-1} \approx \underbrace{0.368}_{\text{share of rows left out}}$$

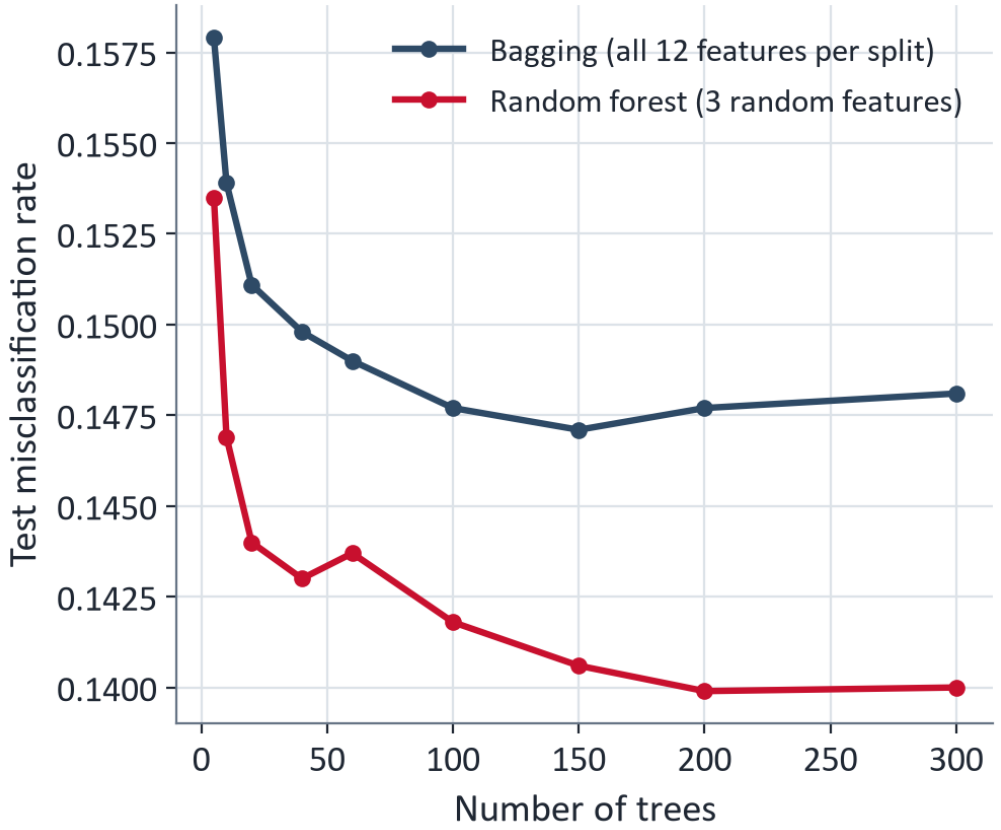
- Each bootstrap sample leaves out about **37 % of the rows**; these rows are *out of bag* (OOB) for that tree.
- The OOB prediction of a row averages **only the trees that did not see it**; the OOB error averages over all rows.
- So OOB gives a **validation estimate for free**: no rows are set aside.
- Adult data, 8,000 rows: OOB error of 300 trees **0.141**, test error **0.140**.
- With few trees the estimate is unstable; tuning many settings on OOB makes it optimistic, so the **test set** gives the final report.



Random forest on the Adult income data

Random forests: decorrelating the trees

- Bagged trees are **correlated**: a strong predictor is chosen at the top of almost every tree.
- A *random forest* (Breiman, 2001) considers only **m randomly chosen predictors** at each split.
  - The default m = √p decorrelates the trees, so the $\rho\sigma^2$ term of the variance falls.
- On the Adult data (p = 12), 300 trees reach test error **0.140** with m = 3, against 0.148 with bagging.
- More trees reduce **Monte Carlo variation** of a fixed forest procedure; held-out error can still fluctuate.
  - Tree count is chosen for stability and cost; depth and leaf size still require validation.
- When few of many predictors are relevant, a small m often **misses them**, and m must be raised.
  - With m = p, a random forest reduces to **bagging**.



Same rows, same seeds, same tree settings

05

Boosting and gradient-boosting libraries

Bagging averages independently constructed trees; boosting optimises an additive model through sequential loss reduction



Boosting in plain words: correct what is still wrong

1. **Start simple**
   predict the average for everyone
2. **Look at the errors**
   residual = actual − prediction
3. **Fit a small tree to the errors**
   it learns where the model is wrong
4. **Add a fraction of it**
   learning rate λ, here 0.5
5. **Repeat**
   each tree fixes part of what is left

Round	Tree predicts	Added λ · tree	Prediction F	Residual y − F
0	—	—	250	50
1	40	+20	270	30
2	28	+14	284	16
3	16	+8	292	8

One flat with true price y = 300 thousand EUR; the tree outputs are illustrative.

- Like **golf**: each stroke is played from where the ball lies now, and the strokes get shorter as the ball nears the hole.
- *Bagging*: trees built **independently**, then averaged. *Boosting*: trees built **one after another**, each depending on the previous ones.

AdaBoost: weighted error and exponential loss

Weighted training

Fit a weak learner h_m(x), with output −1 or +1, on the **weighted** rows.

At the start every row has the same weight **1/n**.

Learner weight

$$\text{err}_m = \sum_i w_i \, \mathbf{1}[y_i \neq h_m(x_i)]$$

$$\alpha_m = \frac{1}{2} \ln \frac{1 - \text{err}_m}{\text{err}_m}$$

Weighted share of wrong rows; a better learner gets a larger say α_m.

Weight update

$$w_i \leftarrow w_i e^{-\alpha_m y_i h_m(x_i)}$$

Wrong rows (y·h = −1) get heavier, correct rows lighter; then the weights are normalised to sum to one.

Computed first round: error 0.325, α = 0.365. Before normalisation, an error receives exp(α) = 1.441; a correct row receives exp(−α) = 0.694.

$$H(x) = \text{sign}\left(\sum_m \alpha_m \, h_m(x)\right)$$

weighted vote of all learners

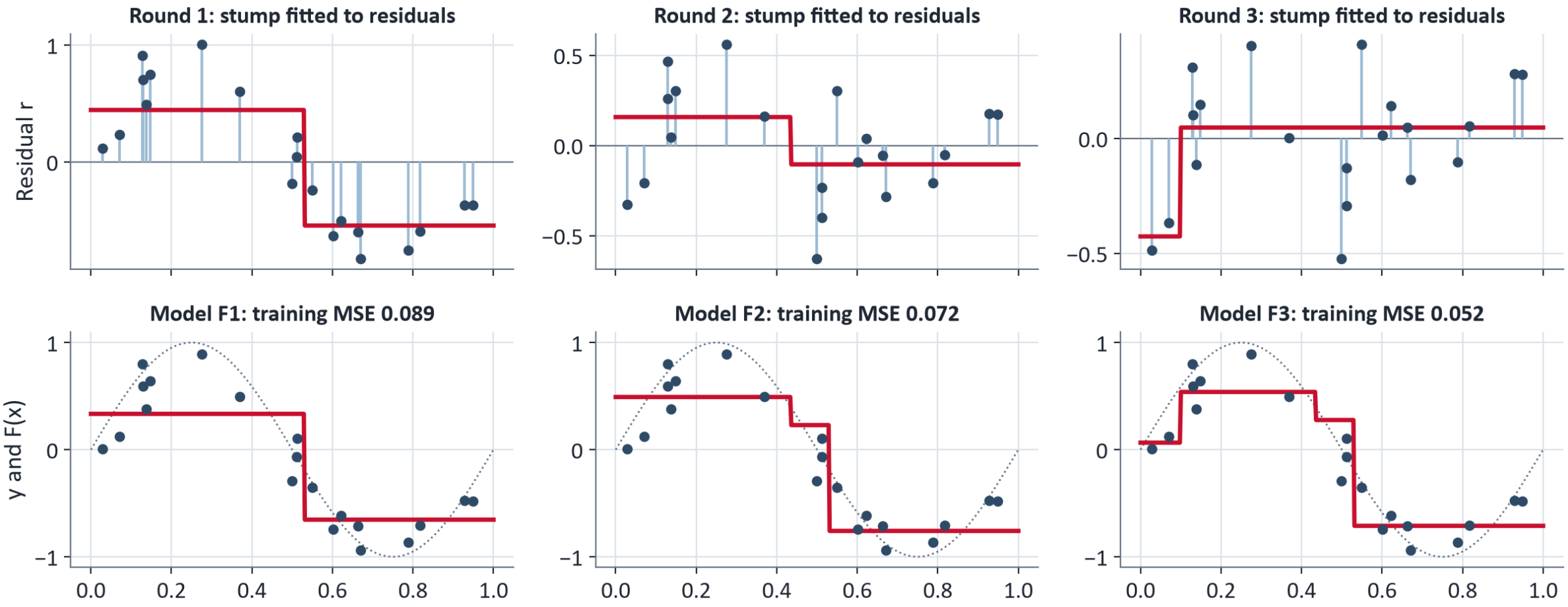
- The final classifier is a **weighted vote**. A useful learner has weighted error below 0.5.
- Rows that stay misclassified get ever larger weights, so **label noise** is a weakness of AdaBoost.

Gradient boosting: fitting the residuals

$$F_m(x) = \underbrace{F_{m-1}(x)}_{\text{model so far}} + \underbrace{\lambda}_{\text{learning rate}} \underbrace{h_m(x)}_{\text{new small tree}} \qquad r_i = \underbrace{y_i - F_{m-1}(x_i)}_{\text{what is still wrong (residual, squared error)}}$$

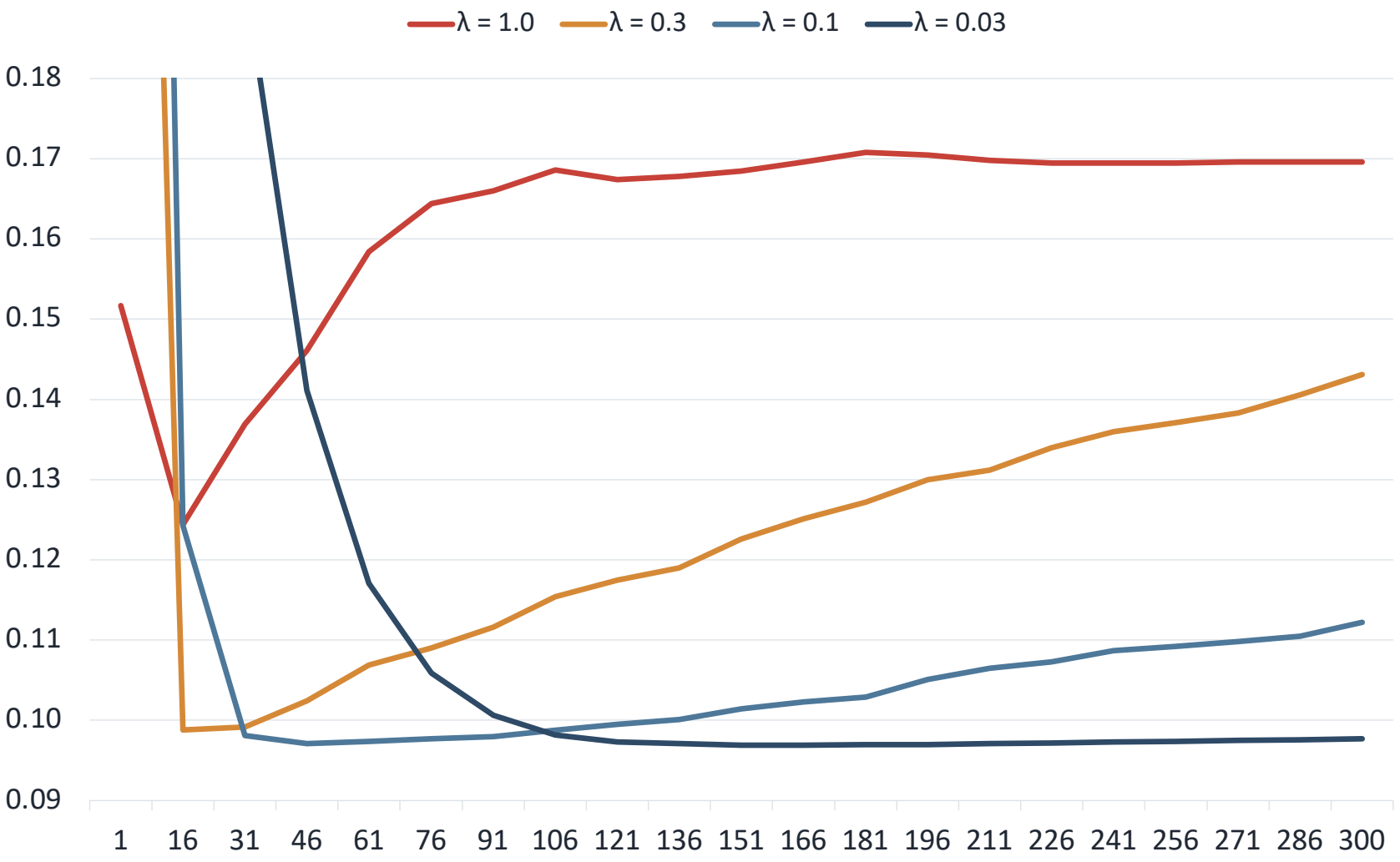
- *Gradient boosting* builds the model **step by step**: each new small regression tree h_m is fitted to the current residuals r_i.
  - In general it fits the *pseudo-residuals*, the negative gradient of the loss; for log loss they are y − p(x).
- This is **gradient descent in function space**: instead of changing parameters, each step adds a function (a tree).
- The *learning rate* λ shrinks each step: **many small corrections** generalise better; their number is chosen on validation data.
- AdaBoost is the special case of exponential loss. Tree depths 3–8 are typical starting values.

Worked example: three rounds of gradient boosting



20 noisy sine points; stumps, λ = 1; training MSE 0.330 → 0.089 → 0.072 → 0.052; top: residual fits; bottom: accumulated model.

The learning rate and the number of trees



- **Independent evaluation MSE** is plotted for **depth-2 trees** fitted to 300 training points.
- With λ = 1 the minimum comes after **6 trees**; then the model overfits.
- With λ = 0.03 it needs 161 trees but reaches **0.0969**.
- These evaluation minima illustrate behaviour, but must not select a model subsequently reported on the same data.
- In practice, **validation loss** selects tree count; 0.01–0.1 is a starting range for λ.

Bagging, random forests and boosting side by side

	Bagging	Random forest	Boosting
How trees are built	Independently, each on a bootstrap sample	Like bagging, plus m random predictors per split	One after another, each fitted to the remaining errors
Trees	Deep	Deep	Small (depth 3–8)
Combination	Average	Average	Sum of small steps (λ)
Mainly reduces	Variance	Variance, more: trees less alike	Bias (and variance through λ)
More trees	Never hurts; only costs time	Never hurts; only costs time	**Can overfit**: choose on validation
Main settings	Number of trees B	B and m (default √p)	Trees, λ and depth
Error estimate	Out-of-bag error	Out-of-bag error	Validation set or CV

Analogy: bagging asks many experts independently and averages; boosting is a team where each new member works on the cases the others got wrong.

06

Naive Bayes

A probabilistic classifier that is estimated by counting, and a baseline for text



Naive Bayes in plain words: a spam filter by counting

From 100 training e-mails	Spam (40)	Not spam (60)
Prior P(class)	40/100 = 0.40	60/100 = 0.60
Contains “free”	20/40 = 0.50	3/60 = 0.05
Contains “meeting”	2/40 = 0.05	24/60 = 0.40

Illustrative counts; only the words that occur in the new e-mail are used.

1. **New e-mail**
   contains “free” and “meeting”
2. **Spam score**
   $0.40 \cdot 0.50 \cdot 0.05 = 0.010$
3. **Not-spam score**
   $0.60 \cdot 0.05 \cdot 0.40 = 0.012$
4. **Decision**
   not spam: $P(\text{spam}) = 0.010 / 0.022 \approx 0.45$

- *Naive*: each word is treated as **independent evidence** given the class, so the probabilities are simply **multiplied**.
- Training is **counting**, with no optimisation: naive Bayes is very fast and a good first model for text.

The naive Bayes classifier

> Naive Bayes assigns x = (x₁, …, x_p) to the class with the largest score:
>
> $$\hat{c} = \arg\max_{c} \underbrace{P(C = c)}_{\text{prior: how common the class is}} \cdot \underbrace{\prod_{j=1}^{p} P(x_j \mid C = c)}_{\text{how typical each predictor value is in that class (independence)}}$$
>
> — Bayes' theorem with the denominator P(x) omitted, because it is the same for every class

- The assumption replaces one joint distribution by **p one-dimensional ones**, estimated separately using distributions appropriate to each feature.
- Categorical x_j use class-wise frequencies; numerical x_j often use **a Gaussian** per class.
- The assumption is rarely true, yet the **ranking of classes** is often still correct.

Estimating the probabilities from training data

Tid	Refund	Status	Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Estimate	Evade = No	Evade = Yes
P(class)	7/10	3/10
P(Refund = Yes | class)	3/7	0/3
P(Status = Single | class)	2/7	2/3
P(Status = Married | class)	4/7	0/3
P(Status = Divorced | class)	1/7	1/3
Income: mean	110	90
Income: sample variance	2,975	25

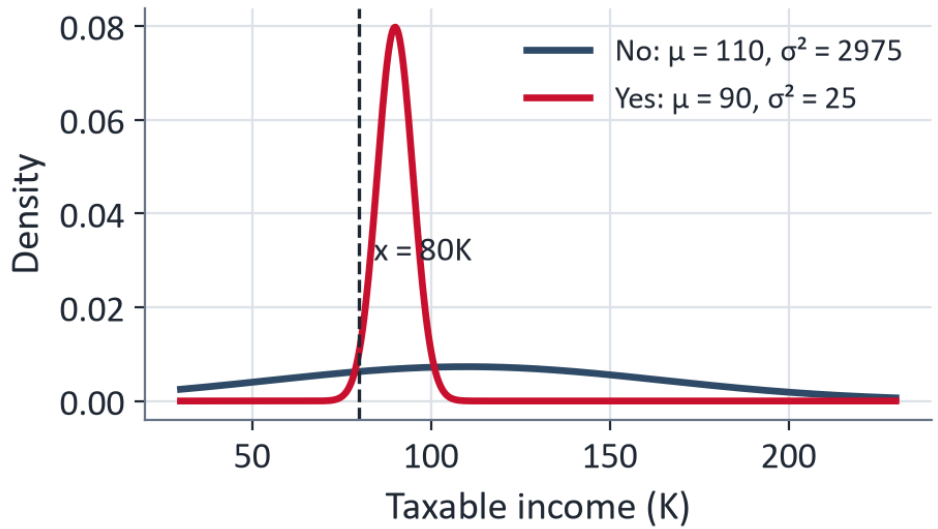
Tan et al. (2019), tax-evasion example. Variances use n_c − 1; GaussianNB uses n_c for maximum-likelihood estimation.

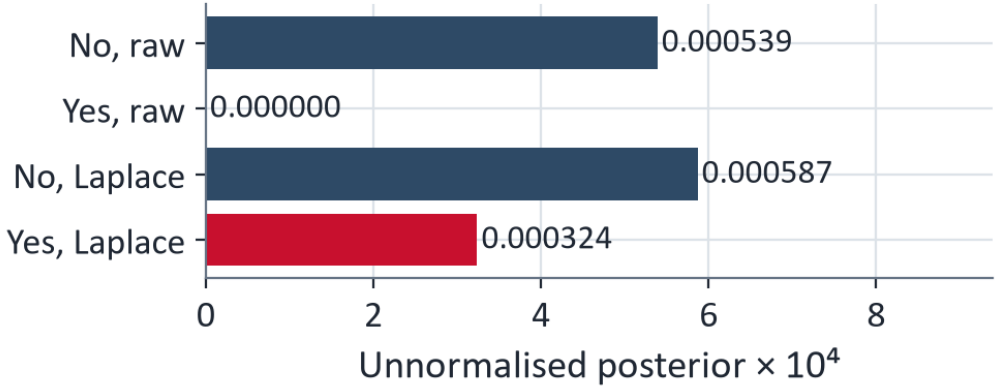
Classifying a record and Laplace smoothing

$$P(x_j = v \mid c) = \frac{\text{count}(v, c) + 1}{n_c + k}$$

add one to every count;
k = number of values

- Record X = (Refund = Yes, Single, 80K); Gaussian densities at 80K: 0.0063 (No), 0.0108 (Yes).
  - No: 0.7 · 3/7 · 2/7 · 0.0063 = **0.000539**.
  - Yes: 0.3 · **0/3** · 2/3 · … = 0: no evader had a refund.
- One **zero count** makes the whole class score zero.
- *Laplace smoothing* (above) removes the zeros: Yes becomes 0.000324, No 0.000587; the decision is still No, but **no longer forced**.
- Normalised: P(No | X) = **0.644**. Implementations **add log-probabilities** to avoid underflow.





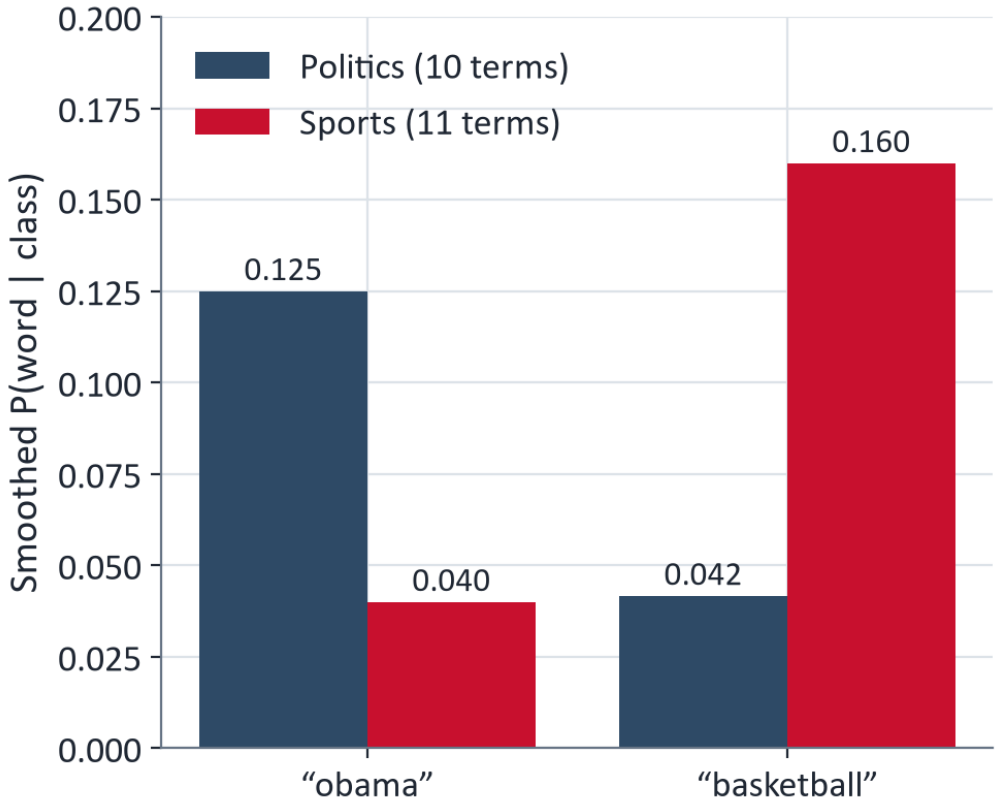
Income densities; unnormalised class scores

Naive Bayes for text classification

$$P(w \mid c) = \underbrace{\frac{\text{count}(w, c) + 1}{N_c + |V|}}_{\text{smoothed share of word w among the words of class c}}$$

$$\text{score}(c) = P(c) \prod_{w} P(w \mid c)^{x_w}$$

- A document is a **bag of words**: x_w counts word w; word order is ignored.
- Training: three politics and three sports titles; vocabulary of **14 words**.
- “Obama likes basketball”: Politics 0.002604, Sports **0.003200**; Sports is chosen. The unknown word “likes” is ignored.
- Fast, needs little data: a **standard baseline for text**; its probabilities need calibration.



Smoothed word likelihoods per class

07

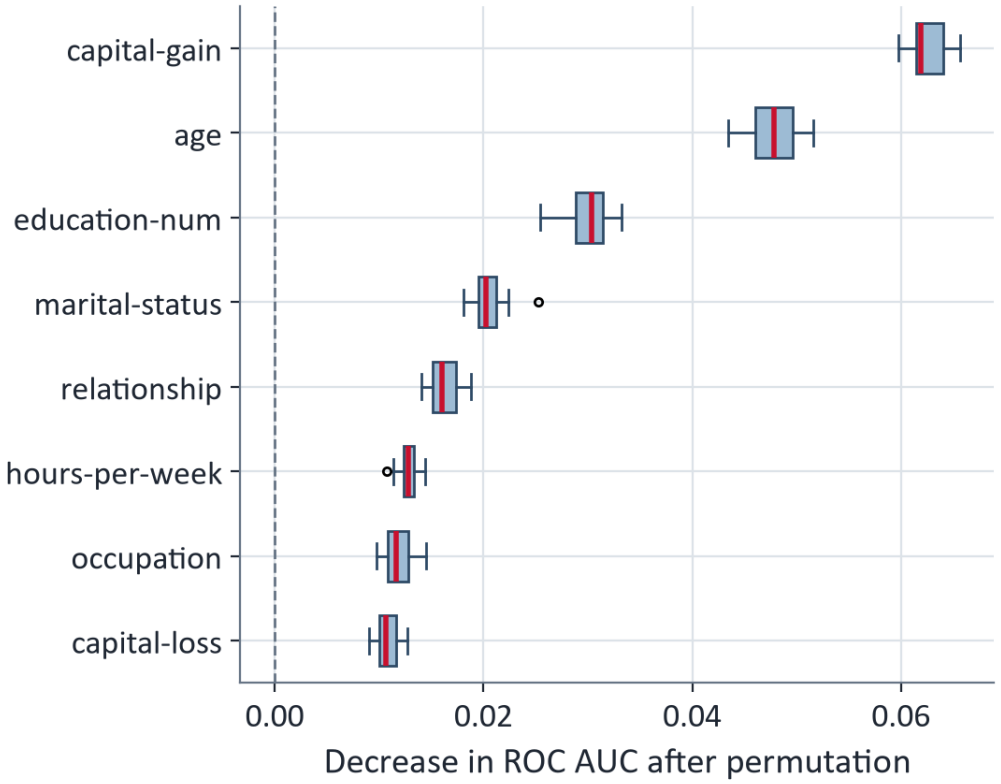
Interpretability and a fair comparison

Complex models require interpretation and evaluation; this section examines predictions and compares five fixed configurations



Permutation importance on held-out data

- *Permutation importance* shuffles one column of **held-out data** and measures the drop in the score.
  - Shuffling breaks the link between the feature and the target and keeps its distribution.
- For HistGradientBoosting, **capital-gain** costs 0.063 ROC AUC and age 0.048.
  - Ten repetitions give **the spread** shown by each box.
- Correlated substitutes can each appear **weak** because the other remains; permutation also creates unrealistic combinations.
- Unlike impurity importance, it works for **any model**, including TabPFN.
  - Importance measures **use by the model**, not causal effect in the population.



HistGradientBoosting, 4,000 test rows (Adult)

Classifier comparison with fold-specific preprocessing

1 The same five folds

Fold 1	**Val.**	Train	Train	Train	Train
Fold 2	Train	**Val.**	Train	Train	Train
Fold 3	Train	Train	**Val.**	Train	Train
Fold 4	Train	Train	Train	**Val.**	Train
Fold 5	Train	Train	Train	Train	**Val.**

Score = mean of the 5 validation scores

2 Inside every fold

Training folds

↓

Fit encoder and scaler

↓

Fit the classifier

↓

Validation fold: apply both

↓

Accuracy and F1

3 Five models, five assumptions

Naive Bayes
independent predictors given the class

Logistic regression
log-odds linear in the predictors

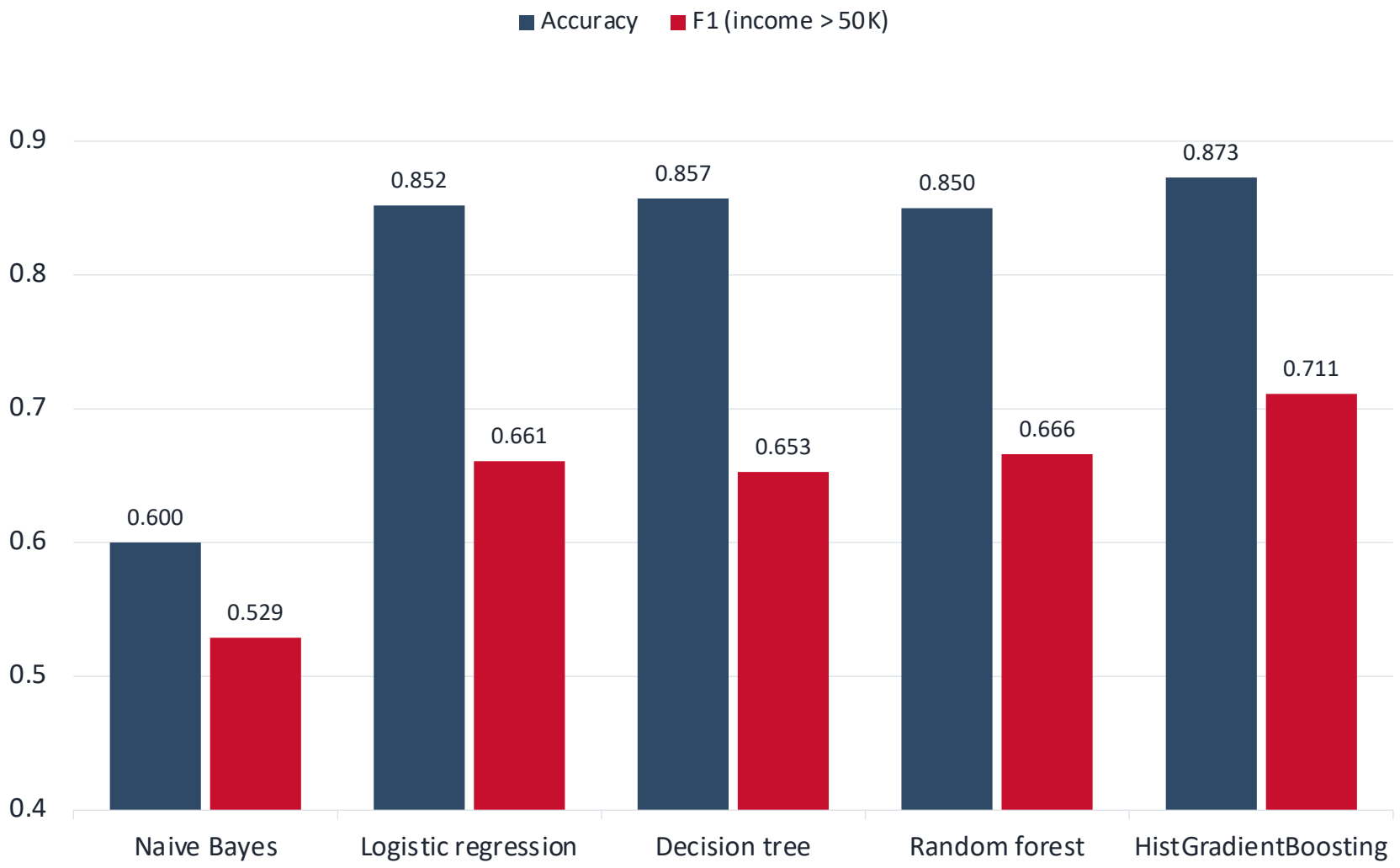
Decision tree, depth 8
axis-aligned rectangles

Random forest, 300 trees
average of decorrelated deep trees

Gradient boosting
sum of many small trees

- This comparison uses **identical folds and metrics** with fixed settings, not equal tuning budgets. Four models fit scaling and one-hot encoding inside training folds; histogram boosting uses categorical columns directly. Tuning needs inner validation; temporal or grouped deployment requires corresponding splits.

Cross-validated accuracy and F1 of five classifiers



- Adult contains **48,842 rows**; the experiment uses shuffled 5-fold stratified CV.
- A constant model scores **0.761** accuracy, so F1 is the more informative measure.
- HistGradientBoosting leads: F1 **0.711** ± 0.006.
- The ± quantity is **fold standard deviation**, not a confidence interval or a statistical significance test.
- Gaussian naive Bayes is **misspecified** for binary one-hot columns; this does not establish failure of all naive Bayes variants.
- The **fixed-configuration ranking** does not establish which family performs best after tuning.

The five classifiers of this lecture: a summary

Method	How it decides	Strengths	Watch out for
Naive Bayes	Multiplies per-feature class probabilities	Very fast; good for text; little data	Independence assumption; poor probabilities
Logistic regression	Linear score → S-curve probability	Probabilities; coefficients read as odds	Linear boundary unless features are added
Decision tree	A sequence of yes/no questions	Easy to explain; no scaling needed	Unstable; overfits unless pruned
Random forest	Average of many decorrelated trees	Strong default; OOB error; little tuning	Larger and slower; harder to explain
Gradient boosting	Sum of small trees, each fixing errors	Often the most accurate on tables	Needs tuning (λ, trees, depth)

Start with a simple baseline (logistic regression or a small tree), then compare stronger models on the same folds and the same metric.

The formulas of this lecture (1/2): logistic regression, metrics

Idea	Formula	In words
Logistic model	$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$	The S-curve turns a linear score into a probability
Log-odds	$\log \frac{p}{1-p} = \beta_0 + \beta_1 x$	Linear in x; one unit of x multiplies the odds by e^β_1
Log loss	$\frac{1}{n}\sum_i \ell_i, \quad \ell_i = -\log p_i \text{ or } -\log(1-p_i)$	Average loss per row (y = 1 or y = 0); punishes confident wrong probabilities
Accuracy, precision	$\frac{TP+TN}{n}, \quad \frac{TP}{TP+FP}$	Share correct; share of the flagged that are right
Recall, F1	$\frac{TP}{TP+FN}, \quad \frac{2PR}{P+R}$	Share of the positives found; balance of P and R

The formulas of this lecture (2/2): trees, ensembles, naive Bayes

Idea	Formula	In words
Gini, entropy	$1 - \sum_k p_k^2, \quad -\sum_k p_k \log_2 p_k$	Impurity of a node: 0 when pure
Information gain	$H(\text{parent}) - \sum_v \frac{n_v}{n} H(v)$	Drop in impurity achieved by a split
Pruning	$R(T) + \alpha \lvert T \rvert$	Training impurity plus a price per leaf
Out of bag	$\left(1 - \frac{1}{n}\right)^n \approx 0.37$	Share of rows a bootstrap sample leaves out
Boosting	$F_m = F_{m-1} + \lambda h_m$	Add a small tree fitted to the residuals
Naive Bayes	$\arg\max_c P(c) \prod_j P(x_j \mid c)$	Most probable class; predictors independent given the class

Key takeaways

- ✓ Logistic regression is **linear in the log-odds** and outputs a probability.
- ✓ A decision tree is a **partition into rectangles**, grown greedily and pruned by cross-validation.
- ✓ Bagging and random forests **reduce variance**; boosting reduces the loss **sequentially**.
- ✓ Naive Bayes is fast and strong on text despite its **independence assumption**.
- ✓ Classifiers are compared on **the same folds and the same metric**; the test set is used once.
- ✓ Importance describes the model, **not the cause**.



Questions

Co-funded by the European Union through the EuroHPC Joint Undertaking under Grant Agreement No 101263128, and by the Ministry of Digital Transformation of the Republic of North Macedonia. Views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the EuroHPC JU.

Vezilka

EuroHPC Joint Undertaking

Co-funded by the European Union

Republic of North Macedonia
Ministry of Digital Transformation